



Two-string consensus problem under non-overlapping inversion and transposition distance

Toan Thang Ta, Cheng-Yao Lin, Chin Lung Lu^{*}

Department of Computer Science, National Tsing Hua University, Hsinchu 30013, Taiwan

ARTICLE INFO

Article history:

Received 7 November 2016

Accepted 22 October 2017

Communicated by R. Uehara

Keywords:

Algorithms

Computational biology

Two-string consensus problem

Inversion

Transposition

ABSTRACT

For biological sequences that can be represented as strings over a finite alphabet, inversion and transposition are commonly observed mutation operations. The non-overlapping inversion and transposition distance (also simply called mutation distance) between two strings is defined as the minimum number of non-overlapping inversion and transposition operations used to transform one string into the other. Given two strings of the same length n and a constant $c \geq 0$, the two-string consensus problem under mutation distance is to determine whether or not there exists a string s^* such that the mutation distance from s^* to each input string does not exceed c . In this study, we present an $O(n^5)$ time and $O(n^4)$ space algorithm to solve this problem.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

Given a set of strings $S = \{s_1, s_2, \dots, s_k\}$ and a constant c , the *consensus (string) problem* is to find, if it exists, a string s^* such that the distance of s^* from each string of S does not exceed c , where s^* is then called the *consensus string* of S . This problem is a fundamental problem in computer science and has also many applications in web searching, computational geometry and computational biology [1,5,8]. In fact, finding the consensus of a given set of strings is a hard and challenging problem. Amir et al. [1] have showed that the consensus problem is NP-hard for the swap metric and APX-hard for the reversal metric. Under the Hamming distance, Frances and Litman [4] have proved that the consensus problem is NP-hard even when the input strings are binary.

In biology, DNA sequences are subject to be changed by genome rearrangements (i.e., genetic mutations at the level of sequence fragments) during their evolutionary process,

such as inversion and transposition [6]. An *inversion* operation substitutes a segment of the DNA sequence by its reverse complement sequence. A *transposition* (also called *translocation* in some literature, e.g., [3]) operation moves a fragment of the DNA sequence from one location to another or, equivalently, exchanges two adjacent and non-overlapping fragments on the DNA sequence. The DNA sequences changed by the genome rearrangements can be inherited to subsequent generations, resulting in new DNA sequences descended from an ancestral sequence. Basically, the DNA sequences descended from a common ancestor can be different because they may evolve with different genetic mutations. Recently, Cho et al. [2] proposed the following problem to study whether or not two DNA sequences are mutated from a common ancestral sequence: given two strings (or sequences), find a *common ancestor* (described as an *alignment* in [2]) for two input strings using non-overlapping inversions. In fact, this problem can be considered as a relaxed version of the consensus problem on two strings under non-overlapping inversion distance, where the constant c is not present as a parameter or, equivalently, $c = \infty$. For this problem, Cho et al. [2] also provided an $O(n^3)$ time algorithm using $O(n^2)$ space, where n is the size of the two input strings. Later, Zohora

^{*} Corresponding author.

E-mail addresses: toanthanghy@gmail.com (T.T. Ta),
begoingto0830@gmail.com (C.-Y. Lin), clu@cs.nthu.edu.tw (C.L. Lu).

and Rahman [9] gave a counterexample to show that the algorithm proposed by Cho et al. [2] is incorrect for some instances. They also designed an algorithm, whose worst-case and average-case time complexities are claimed to be $O(n^4)$ and $O(n^3)$, respectively, to determine the existence of the consensus string under the non-overlapping inversion distance. Very recently, Cho et al. [3] extended their algorithm in [2] to deal with reversals and transpositions, where a *reversal* operation changes a substring by reversing the order of its characters. Note that the transposition used in [3] is restricted to exchange two adjacent and non-overlapping substrings of the same length. The algorithm in [3] has the same time and space complexities as the one from [2]. However, we found that the algorithm proposed by Cho et al. [3] is still incorrect with the counterexample given by Zohora and Rahman in [9].

To address the shortcoming and limitations in the algorithm proposed by Cho et al. [3], we study the consensus problem on two strings (also called *two-string consensus problem*) using non-overlapping inversion and transposition distance in this work. As a result, we devise an algorithm to determine the existence of a consensus string for two input strings in $O(n^5)$ time and $O(n^4)$ space in the worst case. It should be noted that in our study, the threshold c is required as a part of the problem's input, inversion is the reverse and complement of a string, and the lengths of two adjacent and non-overlapping substrings exchanged by a transposition operation can be different (not necessarily equivalent). The rest of the paper is organized as follows. In Section 2, we give the formal definition of the problem. Subsequently, we discuss the main algorithm and its complexity analysis in Section 3. Finally, we have a brief conclusion in Section 4.

2. Preliminaries

Below, we introduce some notations and terminology from [7]. Let z be a string of length n over a finite alphabet Σ . A character at position i of z is represented with z_i (or $z[i]$), where $1 \leq i \leq n$. A substring of z from position i to j is indicated as $z_{i,j}$, i.e., $z_{i,j} = z_i z_{i+1} \dots z_j$, for $1 \leq i \leq j \leq n$. For biological sequences, $\Sigma = \{A, C, G, T\}$, in which A-T and C-G are considered as *complementary base pairs*. We use $\theta(z)$ to denote an *inversion operation* acting on z , resulting in a reverse and complement of z . For example, $\theta(A) = T$, $\theta(T) = A$, $\theta(G) = C$, $\theta(C) = G$ and $\theta(CGA) = TCG$. In addition, we utilize $\tau(uv) = vu$ to represent a *transposition operation* to exchange two non-empty strings u and v . Note that the lengths of u and v may be different in this study. For convenience, we call θ and τ as *mutation operations*. We also let $\theta_{i,j}(z) = \theta(z_{i,j})$ for $1 \leq i \leq j \leq n$ and $\tau_{i,j,k}(z) = z_{k,j} z_{i,k-1}$ for $1 \leq i < k \leq j \leq n$, where $[i, j]$ is called a *mutation range* for $\theta_{i,j}$ and $\tau_{i,j,k}$. For an integer t , where $1 \leq t \leq n$, we say that a mutation operation $\theta_{i,j}$ or $\tau_{i,j,k}$ covers t if $i \leq t \leq j$. Given two mutation operations, they are *non-overlapping* if the intersection of their mutation ranges is empty. Given a set Θ of non-overlapping mutation operations and a string z , let $\Theta(z)$ be the resulting string after consecutively applying mutation operations in Θ on z . For example, if $\Theta = \{\tau_{1,3,2}, \theta_{5,5}\}$ and $z = TAGAC$, then $\Theta(z) = AGTAG$.

Definition 1 (*Non-overlapping inversion and transposition distance*). Given two strings x and y of the same length, the non-overlapping inversion and transposition distance (simply called *mutation distance*) between x and y , denoted by $md(x, y)$, is defined as the minimum number of non-overlapping inversion and transposition operations used to transform x into y . If there does not exist any set of non-overlapping mutations that converts x into y , then $md(x, y)$ is infinite. Formally,

$$md(x, y) = \begin{cases} \min \{|\Theta| : \Theta(x) = y\} & \text{if } \exists \Theta \text{ such that} \\ & \Theta(x) = y \\ \infty & \text{otherwise} \end{cases}$$

Definition 2 (*Two-string consensus problem under mutation distance*).

Input: Two strings x and y of the same length n and a constant $c \geq 0$.

Question: Does there exist a string s^* such that $md(s^*, x) \leq c$ and $md(s^*, y) \leq c$?

For example, suppose that $x = TAGAC$ and $y = TAACG$. When $c = 1$, the answer is positive because there exists $s^* = y$ such that $md(s^*, x) = 1$ and $md(s^*, y) = 0$. However, when $c = 0$, the answer is negative, since $x \neq y$ and hence there does not exist any s^* satisfying both $md(s^*, x) = 0$ and $md(s^*, y) = 0$.

3. Our algorithm

In fact, a mutation acting on a string z comprises several sub-operations, called *mutation fragments*, each of which is denoted either by a tuple (i, j, z_j) or $(i, j, \theta(z_j))$, where (i, j, z_j) (respectively, $(i, j, \theta(z_j))$) means that z_j (respectively, complement of z_j) is moved into the position i in the resulting string obtained when applying the mutation operation on z . For convenience, we arrange all possible mutation fragments for z in two *mutation tables* M_1^z and M_2^z , where $M_1^z[i, j] = (i, j, \theta(z_j))$ and $M_2^z[i, j] = (i, j, z_j)$ for all $1 \leq i, j \leq n$. For example, if $z = TAGAC$, then its mutation tables are shown in Fig. 1.

Given a string z and its mutation tables M_1^z and M_2^z , the result of an inversion $\theta_{i,j}(z)$ can be obtained by concatenating $(j - i + 1)$ mutation fragments starting at $M_1^z[i, j]$ and continuing to move in the anti-diagonal direction to $M_1^z[j, i]$ (see the shaded entries of M_1^z in Fig. 1 for an example). That is, we can decompose $\theta_{i,j}(z)$ into $(j - i + 1)$ mutation fragments $\mathcal{F}(\theta_{i,j}, z, t)$ for $i \leq t \leq j$, where $\mathcal{F}(\theta_{i,j}, z, t) = (t, i + j - t, \theta(z_{i+j-t}))$. In addition, the result of a transposition $\tau_{i,j,k}(z)$ can be obtained by concatenating the mutation fragments on the following two paths, one starting at $M_2^z[i, k]$ and continuing to move in the diagonal direction to $M_2^z[i + j - k, j]$ and the other beginning at $M_2^z[i + j - k + 1, i]$ and also continuing to move in the diagonal direction to $M_2^z[j, k - 1]$ (see the shaded entries of M_2^z in Fig. 1 for an example). More clearly, $\tau_{i,j,k}(z)$ can be decomposed into $(j - i + 1)$ mutation fragments $\mathcal{F}(\tau_{i,j,k}, z, t)$ for $i \leq t \leq j$, where if $i \leq t \leq i + j - k$, then $\mathcal{F}(\tau_{i,j,k}, z, t) = (t, k + t - i, z_{k+t-i})$; otherwise, $\mathcal{F}(\tau_{i,j,k}, z, t) = (t, t + j + k - 1, z_{t-j+k-1})$. For the purpose of brevity, we further let $\tau_{i,j,k}(z, 1) = \{(t, k + t - i,$

$M_1^z[i, j]$	$i = 1$	2	3	4	5
$j = 1$	(1,1,A)	(2,1,A)	(3,1,A)	(4,1,A)	(5,1,A)
2	(1,2,T)	(2,2,T)	(3,2,T)	(4,2,T)	(5,2,T)
3	(1,3,C)	(2,3,C)	(3,3,C)	(4,3,C)	(5,3,C)
4	(1,4,T)	(2,4,T)	(3,4,T)	(4,4,T)	(5,4,T)
5	(1,5,G)	(2,5,G)	(3,5,G)	(4,5,G)	(5,5,G)

$M_2^z[i, j]$	$i = 1$	2	3	4	5
$j = 1$	(1,1,T)	(2,1,T)	(3,1,T)	(4,1,T)	(5,1,T)
2	(1,2,A)	(2,2,A)	(3,2,A)	(4,2,A)	(5,2,A)
3	(1,3,G)	(2,3,G)	(3,3,G)	(4,3,G)	(5,3,G)
4	(1,4,A)	(2,4,A)	(3,4,A)	(4,4,A)	(5,4,A)
5	(1,5,C)	(2,5,C)	(3,5,C)	(4,5,C)	(5,5,C)

Fig. 1. Mutation tables M_1^z and M_2^z for a given string $z = TAGAC$, where the column is indexed by i and the row by j . Shaded cells respectively represent the inversion $\theta_{1,3}(z)$ on M_1^z and the transposition operation $\tau_{1,5,4}(z)$ on M_2^z .

$z_{k+t-i} : i \leq t \leq i + j - k$ and $\tau_{i,j,k}(z, 2) = \{(t, t - j + k - 1, z_{t-j+k-1}) : i + j - k < t \leq j\}$.

The aforementioned decomposition can be extended to apply on a set Θ of non-overlapping mutation operations. That is, when Θ acts on a string z , Θ can be decomposed into a sequence $\mathcal{F}(\Theta, z) = \langle F_1, F_2, \dots, F_n \rangle$ of mutation fragments by the following formula: For $t = 1, 2, \dots, n$,

$$F_t = \mathcal{F}(\Theta, z, t) = \begin{cases} \mathcal{F}(\omega, z, t) & \text{if } \exists \omega \in \Theta \text{ that covers } t \\ (t, t, z_t) & \text{otherwise} \end{cases}$$

For instance, consider $\Theta = \{\tau_{1,2,2}, \theta_{4,5}\}$ as well as a string $z = TAGAC$. Then we have $\mathcal{F}(\Theta, z) = \langle (1, 2, A), (2, 1, T), (3, 3, G), (4, 5, G), (5, 4, T) \rangle$.

For a mutation fragment $F = (i, j, \sigma)$, we say that F yields the character σ , denoted by $\ell(F) = \sigma$. Given a mutation operation $\theta_{i,j}$ or $\tau_{i,j,k}$, we call i and j as the *left end point* and *right end point* of the mutation operation, respectively. We also say that this mutation operation (i.e., $\theta_{i,j}$ or $\tau_{i,j,k}$) intersects with a range $[a, b]$ if the intersection of $[i, j]$ and $[a, b]$ is non-empty. The number of mutation operations in Θ that intersects with the range $[1, m]$ where $m \leq n$ is denoted as $\rho(\Theta, m)$. For example, if $\Theta = \{\theta_{1,2}, \tau_{4,6,5}, \theta_{7,9}\}$, then $\rho(\Theta, 4) = 2$. For an integer i , let $A_z^i = \{M_1^z[i+1, t] : i+1 < t \leq n\}$ and $B_z^i = \{M_2^z[i+1, t] : i+1 < t \leq n\}$, which are sets of all mutation fragments covered by inversion and transposition operations respectively acting on z whose left end points are all $i+1$, where $0 \leq i < n$.

Definition 3 (Matched pair). Let Θ_x and Θ_y be two sets of non-overlapping mutation operations acting on strings x and y , respectively. We say that Θ_x and Θ_y form a *matched pair of span length m* if $\rho(\Theta_x, m) \leq c$, $\rho(\Theta_y, m) \leq c$ and $\ell(\mathcal{F}(\Theta_x, x, t)) = \ell(\mathcal{F}(\Theta_y, y, t))$ for all $1 \leq t \leq m$.

The main idea of our algorithm is as follows. First, we have only a matched pair of span length 0. Next, for each $0 \leq i \leq n-1$, the algorithm produces all possible matched pairs of span length $i+1$, which can be obtained from all previously created matched pairs of span length i . Finally, the algorithm returns “yes” if there exists a matched pair of span length n . Otherwise, the algorithm returns “no”.

Suppose that we are given any set Θ of non-overlapping mutation operations acting on a string z of length n . In the following, we define four sets of *extended* mutation fragments I_z^i , H_z^i , T_z^i and Q_z^i to represent the possible “status” of Θ at the position i in z .

- $I_z^i = \{(g, d, M_1^z[i, j])\}$ for $0 \leq g \leq n-1$, $0 \leq d \leq n$ and $1 \leq j \leq n$.
- $H_z^i = \{(s, d, M_2^z[i, j])\}$ for $1 \leq s \leq n-1$, $0 \leq d \leq n$ and $i < j \leq n$.
- $T_z^i = \{(g, d, M_2^z[i, j])\}$ for $0 \leq g \leq n-1$, $0 \leq d \leq n$ and $1 \leq j < i$.
- $Q_z^i = \{(0, d, M_2^z[i, i])\}$ for $0 \leq d \leq n$.

Actually, each extended mutation fragment defined above contains a mutation fragment at column i of the mutation tables M_1^z and M_2^z with extra guiding information g, d and s . The value of g is the number of steps that are required to move towards the anti-diagonal (respectively, diagonal) direction on M_1^z (respectively, M_2^z) to complete an inversion (respectively, transposition) operation. The value of s is the left end point of the mutation operation in Θ covering position i and the value of d indicates the number of mutation operations in Θ that intersect with the range $[1, i]$. If Θ contains a mutation operation ω covering position i in z , then we use $\mathcal{L}(\omega)$ and $\mathcal{R}(\omega)$ to denote the left and right end points of ω , respectively; otherwise, we still say that position i is covered by a *virtual mutation operation* for convenience. In other words, every position i in z can be covered by a (virtual) mutation operation. Then the *status* of Θ acting on string z at position i , denoted by $\mu(\Theta, z, i)$, is defined as follows according to four different cases (each case is called as a *status type*), where we let $F_i = \mathcal{F}(\Theta, z, i)$:

Case 1 (type I): ω is an inversion operation. Then $\mu(\Theta, z, i) = (\mathcal{R}(\omega) - i, \rho(\Theta, i), F_i)$.

Case 2 (type H): ω is a transposition operation and $F_i \in \omega(z, 1)$. Then $\mu(\Theta, z, i) = (\mathcal{L}(\omega), \rho(\Theta, i), F_i)$.

Case 3 (type T): ω is a transposition operation and $F_i \in \omega(z, 2)$. Then $\mu(\Theta, z, i) = (\mathcal{R}(\omega) - i, \rho(\Theta, i), F_i)$.

Case 4 (type Q): ω is a virtual operation. Then $\mu(\Theta, z, i) = (0, \rho(\Theta, i), F_i)$.

We say that Θ stops at position i if ω is a virtual operation or $\mathcal{R}(\omega) = i$ (i.e., the first component in $\mu(\Theta, z, i)$ is zero); otherwise, we say that Θ continues at i . If Θ stops at i , then its status $\mu(\Theta, z, i)$ is called an *ending fragment*; otherwise, $\mu(\Theta, z, i)$ is called a *continuing fragment*. We use $d(\mu(\Theta, z, i))$ to denote the value of the second component in $\mu(\Theta, z, i)$, i.e., $d(\mu(\Theta, z, i)) = \rho(\Theta, i)$.

Observation 1 ([7]). Let Θ be a set of non-overlapping mutation operations acting on a string z of length n . If Θ stops at position i on z , then $\mathcal{F}(\Theta, z, i+1) \in A_z^i \cup B_z^i \cup M_z^i[i+1, i+1]$.

In order to represent all possible matched pairs of span length i , we construct a bipartite graph $G_i = (U_i, V_i, E_i)$ as follows:

- $U_i = I_x^i \cup H_x^i \cup T_x^i \cup Q_x^i$.
- $V_i = I_y^i \cup H_y^i \cup T_y^i \cup Q_y^i$.
- Given a matched pair Θ_x and Θ_y of span length i , let $u = \mu(\Theta_x, x, i) \in U_i$ and $v = \mu(\Theta_y, y, i) \in V_i$. If $d(u) \leq c$ and $d(v) \leq c$ (i.e., $\rho(\Theta_x, i) \leq c$ and $\rho(\Theta_y, i) \leq c$), then we create an edge (u, v) in E_i .

An edge (u, v) in G_i is classified into one of 16 types $II, IH, IT, IQ, HI, HH, HT, HQ, TI, TH, TT, TQ, QI, QH, QT$ and QQ that correspond to all combinations of status types of u and v . Now, we describe our algorithm to solve the two-string consensus problem. Initially, G_0 contains only one edge (u, v) of type QQ corresponding to the matched pair of span length 0, where $d(u) = 0$ and $d(v) = 0$. Assume that $G_i = (U_i, V_i, E_i)$ is already created, i.e., all matched pairs of span length i are known. The algorithm then constructs $G_{i+1} = (U_{i+1}, V_{i+1}, E_{i+1})$ by examining all edges in E_i . The algorithm processes each edge $(u, v) \in E_i$ according to its type as follows:

Case 1: (u, v) of type II . Suppose that $u = (g_x, d_x, M_1^x[i, j_x])$ and $v = (g_y, d_y, M_1^y[i, j_y])$.

Case 1.1: Both u and v are continuing fragments, i.e., $g_x > 0$ and $g_y > 0$. If $\theta(x[j_x - 1]) = \theta(y[j_y - 1])$, then we create an edge (u', v') of type II in G_{i+1} where $u' = (g_x - 1, d_x, M_1^x[i+1, j_x - 1])$ and $v' = (g_y - 1, d_y, M_1^y[i+1, j_y - 1])$.

Case 1.2: One of two extended mutation fragments u or v is an ending fragment. If u is an ending fragment, i.e., $g_x = 0$ and $g_y > 0$ (or v is an ending fragment, i.e., $g_x > 0$ and $g_y = 0$), then we simply change the type of (u, v) into QI (or IQ , respectively) in G_i .

Case 1.3: Both u and v are ending fragments, i.e., $g_x = 0$ and $g_y = 0$. We change the type of (u, v) into QQ in G_i .

Case 2: (u, v) of type IH . Suppose that $u = (g_x, d_x, M_1^x[i, j_x])$ and $v = (s_y, d_y, M_2^y[i, j_y])$.

Case 2.1: u is a continuing fragment, i.e., $g_x > 0$. We have two tasks to complete. First, if $j_y < n$ and $\theta(x[j_x - 1]) = y[j_y + 1]$, then we create an edge (u', v') of type IH in G_{i+1} where $u' = (g_x - 1, d_x, M_1^x[i+1, j_x - 1])$ and $v' = (s_y, d_y, M_2^y[i+1, j_y + 1])$. Second, if $\theta(x[j_x - 1]) = y[s_y]$, then we create an edge (u', v') of type IT in G_{i+1} where $u' = (g_x - 1, d_x, M_1^x[i+1, j_x - 1])$ and $v' = (j_y - i - 1, d_y, M_2^y[i+1, s_y])$.

Case 2.2: u is an ending fragment. We simply change the type of (u, v) into QH in G_i .

Case 3: (u, v) of type IT . Suppose that $u = (g_x, d_x, M_1^x[i, j_x])$ and $v = (g_y, d_y, M_2^y[i, j_y])$.

Case 3.1: Both u and v are continuing fragments, i.e., $g_x > 0$ and $g_y > 0$. If $\theta(x[j_x - 1]) = y[j_y + 1]$, then we create an edge (u', v') of type IT in G_{i+1} where $u' = (g_x - 1, d_x, M_1^x[i+1, j_x - 1])$ and $v' = (g_y - 1, d_y, M_2^y[i+1, j_y + 1])$.

Case 3.2: One of two extended mutation fragments u or v is an ending fragment. If u is an ending fragment, i.e., $g_x = 0$ and $g_y > 0$ (or v is an ending fragment, i.e., $g_x > 0$ and $g_y = 0$, respectively). We simply change the type of (u, v) into QT (or IQ , respectively) in G_i .

Case 3.3: Both u and v are ending fragments, i.e., $g_x = 0$ and $g_y = 0$. We change the type of (u, v) into QQ in G_i .

Case 4: (u, v) of type HH . Suppose that $u = (s_x, d_x, M_2^x[i, j_x])$ and $v = (s_y, d_y, M_2^y[i, j_y])$. There are four tasks to complete. First, if $j_x < n$, $j_y < n$ and $x[j_x + 1] = y[j_y + 1]$, then we create an edge (u', v') of type HH in G_{i+1} where $u' = (s_x, d_x, M_2^x[i+1, j_x + 1])$ and $v' = (s_y, d_y, M_2^y[i+1, j_y + 1])$. Second, if $j_x < n$ and $x[j_x + 1] = y[s_y]$, then we create an edge (u', v') of type HT in G_{i+1} where $u' = (s_x, d_x, M_2^x[i+1, j_x + 1])$ and $v' = (j_y - i - 1, d_y, M_2^y[i+1, s_y])$. Third, if $j_y < n$ and $x[s_x] = y[j_y + 1]$, then we create an edge (u', v') of type TH in G_{i+1} where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (s_y, d_y, M_2^y[i+1, j_y + 1])$. Finally, if $x[s_x] = y[s_y]$, then we create an edge (u', v') of type TT in G_{i+1} where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (j_y - i - 1, d_y, M_2^y[i+1, s_y])$.

Case 5: (u, v) of type HT . Suppose that $u = (s_x, d_x, M_2^x[i, j_x])$ and $v = (g_y, d_y, M_2^y[i, j_y])$.

Case 5.1: v is a continuing fragment, i.e., $g_y > 0$. There are two tasks to complete. First, if $j_x < n$ and $x[j_x + 1] = y[j_y + 1]$, then we create an edge (u', v') of type HT in G_{i+1} where $u' = (s_x, d_x, M_2^x[i+1, j_x + 1])$ and $v' = (g_y - 1, d_y, M_2^y[i+1, j_y + 1])$. Second, if $x[s_x] = y[j_y + 1]$, we create an edge (u', v') of type TT in G_{i+1} where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (g_y - 1, d_y, M_2^y[i+1, j_y + 1])$.

Case 5.2: v is an ending fragment, i.e., $g_y = 0$. We change the type of (u, v) into HQ in G_i .

Case 6: (u, v) of type TT . Suppose that $u = (g_x, d_x, M_2^x[i, j_x])$ and $v = (g_y, d_y, M_2^y[i, j_y])$.

Case 6.1: Both u and v are continuing fragments, i.e., $g_x > 0$ and $g_y > 0$. If $x[j_x + 1] = y[j_y + 1]$, then we create an edge (u', v') of type TT in G_{i+1} where $u' = (g_x - 1, d_x, M_2^x[i+1, j_x + 1])$ and $v' = (g_y - 1, d_y, M_2^y[i+1, j_y + 1])$.

Case 6.2: One of two extended mutation fragments u or v is an ending fragment. If u is an ending fragment, i.e., $g_x = 0$ and $g_y > 0$ (or v is an ending fragment, i.e., $g_x > 0$ and $g_y = 0$, respectively). We simply change the type of (u, v) into QT (or TQ , respectively) in G_i .

Case 6.3: Both u and v are ending fragments, i.e., $g_x = 0$ and $g_y = 0$. We change the type of (u, v) into QQ in G_i .

When (u, v) is of type HI, TI and TH , we process it as of type IH, IT and HT , respectively. For remaining cases (IQ, HQ, TQ, QI, QH, QT and QQ), we process them as directly stated in [Procedure 2](#).

Below, we detail our method in [Algorithm 1](#) based on the idea mentioned above.

Algorithm 1. Two-string consensus problem under non-overlapping inversion and transposition distance.

Input: Two strings x and y of the same length n and constant $c \geq 0$.

Output: Determine whether or not there exists a string s^* such that both $md(s^*, x)$ and $md(s^*, y)$ do not exceed c .

1. Let G_0 be a bipartite graph that has only one edge (u, v) of type $Q Q$, where $d(u) = 0$ and $d(v) = 0$;
2. **for** $i = 1$ **to** $n - 1$ **do**
3. $G_{i+1} = \text{GraphExtension}(G_i, c)$;
4. **if** there is no edge in G_{i+1} **then** return *false*;
5. Return *true*;

Procedure 1. EdgeCreation(i, d_x, d_y, c).

Input: Position i , current numbers of mutation operations d_x and d_y , and constant $c \geq 0$.

Output: Create edges in G_{i+1} when both extended mutation fragments are ending fragments at position i .

1. **for** each $M_1^x[i+1, q] \in A_x^i$ **do**
2. **for** each $M_1^y[i+1, t] \in A_y^i$ **do**
3. **if** $\theta(x[q]) = \theta(y[t])$ and $d_x < c$ and $d_y < c$ **then**
4. Create an edge (u, v) of type II in G_{i+1} , where $u = (q - i - 1, d_x + 1, M_1^x[i+1, q])$ and $v = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
5. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
6. **if** $\theta(x[q]) = y[t]$ and $d_x < c$ and $d_y < c$ **then**
7. Create an edge (u, v) of type IH in G_{i+1} , where $u = (q - i - 1, d_x + 1, M_1^x[i+1, q])$ and $v = (i + 1, d_y + 1, M_2^y[i+1, t])$;
8. **if** $\theta(x[q]) = y[i+1]$ and $d_x < c$ **then**
9. Create an edge (u, v) of type IQ in G_{i+1} , where $u = (q - i - 1, d_x + 1, M_1^x[i+1, q])$ and $v = (0, d_y, M_2^y[i+1, i+1])$;
10. **for** each $M_2^x[i+1, q] \in B_x^i$ **do**
11. **for** each $M_1^y[i+1, t] \in A_y^i$ **do**
12. **if** $x[q] = \theta(y[t])$ and $d_x < c$ and $d_y < c$ **then**
13. Create an edge (u, v) of type HI in G_{i+1} , where $u = (i + 1, d_x + 1, M_2^x[i+1, q])$ and $v = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
14. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
15. **if** $x[q] = y[t]$ and $d_x < c$ and $d_y < c$ **then**
16. Create an edge (u, v) of type HH in G_{i+1} , where $u = (i + 1, d_x + 1, M_2^x[i+1, q])$ and $v = (i + 1, d_y + 1, M_2^y[i+1, t])$;
17. **if** $x[q] = y[i+1]$ and $d_x < c$ **then**
18. Create an edge (u, v) of type HQ in G_{i+1} , where $u = (i + 1, d_x + 1, M_2^x[i+1, q])$ and $v = (0, d_y, M_2^y[i+1, i+1])$;
19. **for** each $M_1^y[i+1, t] \in A_y^i$ **do**
20. **if** $x[i+1] = \theta(y[t])$ and $d_y < c$ **then**
21. Create an edge (u, v) of type QI in G_{i+1} , where $u = (0, d_x, M_2^x[i+1, i+1])$ and $v = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
22. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
23. **if** $x[i+1] = y[t]$ and $d_y < c$ **then**

24. Create an edge (u, v) of type QH in G_{i+1} , where $u = (0, d_x, M_2^x[i+1, i+1])$ and $v = (i + 1, d_y + 1, M_2^y[i+1, t])$;
25. **if** $x[i+1] = y[i+1]$ **then**
26. Create an edge (u, v) of type QQ in G_{i+1} , where $u = (0, d_x, M_2^x[i+1, i+1])$ and $v = (0, d_y, M_2^y[i+1, i+1])$;

Procedure 2. GraphExtension(G_i, c).

Input: G_i and constant $c \geq 0$.

Output: G_{i+1} .

1. Process the edges of type $II, IH, IT, HI, HH, HT, TI, TH$ and TT in G_i as described in the text;
2. **for** each edge (u, v) of type HQ in G_i **do**
3. Let $u = (s_x, d_x, M_2^x[i, j_x])$ and $v = (0, d_y, M_2^y[i, i])$;
4. **if** $j_x < n$ **then**
5. **for** each $M_2^y[i+1, t] \in A_y^i$ **do**
6. **if** $x[j_x+1] = \theta(y[t])$ and $d_y < c$ **then**
7. Create an edge (u', v') of type HI in G_{i+1} , where $u' = (s_x, d_x, M_2^x[i+1, j_x+1])$ and $v' = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
8. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
9. **if** $x[j_x+1] = y[t]$ and $d_y < c$ **then**
10. Create an edge (u', v') of type HH in G_{i+1} , where $u' = (s_x, d_x, M_2^x[i+1, j_x+1])$ and $v' = (i + 1, d_y + 1, M_2^y[i+1, t])$;
11. **if** $x[j_x+1] = y[i+1]$ **then**
12. Create an edge (u', v') of type HQ in G_{i+1} , where $u' = (s_x, d_x, M_2^x[i+1, j_x+1])$ and $v' = (0, d_y, M_2^y[i+1, i+1])$;
13. **for** each $M_1^y[i+1, t] \in A_y^i$ **do**
14. **if** $x[s_x] = \theta(y[t])$ and $d_y < c$ **then**
15. Create an edge (u', v') of type TI in G_{i+1} , where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
16. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
17. **if** $x[s_x] = y[t]$ and $d_y < c$ **then**
18. Create an edge (u', v') of type TH in G_{i+1} , where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (i + 1, d_y + 1, M_2^y[i+1, t])$;
19. **if** $x[s_x] = y[i+1]$ **then**
20. Create an edge (u', v') of type TQ in G_{i+1} , where $u' = (j_x - i - 1, d_x, M_2^x[i+1, s_x])$ and $v' = (0, d_y, M_2^y[i+1, i+1])$;
21. The process of type QH is similar to that of HQ .
22. **for** each edge (u, v) of type TQ in G_i **do**
23. Let $u = (g_x, d_x, M_2^x[i, j_x])$ and $v = (0, d_y, M_2^y[i, i])$;
24. **case 1:** $g_x > 0$
25. **for** each $M_1^y[i+1, t] \in A_y^i$ **do**
26. **if** $x[j_x+1] = \theta(y[t])$ and $d_y < c$ **then**
27. Create an edge (u', v') of type TI in G_{i+1} , where $u' = (g_x - 1, d_x, M_2^x[i+1, j_x+1])$ and $v' = (t - i - 1, d_y + 1, M_1^y[i+1, t])$;
28. **for** each $M_2^y[i+1, t] \in B_y^i$ **do**
29. **if** $x[j_x+1] = y[t]$ and $d_y < c$ **then**
30. Create an edge (u', v') of type TH in G_{i+1} , where $u' = (g_x - 1, d_x, M_2^x[i+1, j_x+1])$ and $v' = (i + 1, d_y + 1, M_2^y[i+1, t])$;
31. **if** $x[j_x+1] = y[i+1]$ **then**


```

32.   Create an edge  $(u', v')$  of type  $TQ$  in  $G_{i+1}$ ,
      where  $u' = (g_x - 1, d_x, M_2^x[i + 1, j_x + 1])$  and
       $v' = (0, d_y, M_2^y[i + 1, i + 1])$ ;
33.   case 2:  $g_x = 0$ 
34.   Change the type of  $(u, v)$  into  $QQ$  in  $G_i$ ;
35.   The process of type  $QT$  is similar to that in  $TQ$ .
36.   for each edge  $(u, v)$  of type  $IQ$  in  $G_i$  do
37.     Let  $u = (g_x, d_x, M_1^x[i, j_x])$  and  $v = (0, d_y, M_2^y[i, i])$ ;
38.     case 1:  $g_x > 0$ 
39.     for each  $M_1^y[i + 1, t] \in A_y^i$  do
40.       if  $\theta(x[j_x - 1]) = \theta(y[t])$  and  $d_y < c$  then
41.         Create an edge  $(u', v')$  of type  $II$  in  $G_{i+1}$ ,
         where  $u' = (g_x - 1, d_x, M_1^x[i + 1, j_x - 1])$  and
          $v' = (t - i - 1, d_y + 1, M_1^y[i + 1, t])$ ;
42.       for each  $M_2^y[i + 1, t] \in B_y^i$  do
43.         if  $\theta(x[j_x - 1]) = y[t]$  and  $d_y < c$  then
44.           Create an edge  $(u', v')$  of type  $IH$  in  $G_{i+1}$ ,
           where  $u' = (g_x - 1, d_x, M_1^x[i + 1, j_x - 1])$  and
            $v' = (i + 1, d_y + 1, M_2^y[i + 1, t])$ ;
45.         if  $\theta(x[j_x - 1]) = y[i + 1]$  then
46.           Create an edge  $(u', v')$  of type  $IQ$  in  $G_{i+1}$ ,
           where  $u' = (g_x - 1, d_x, M_1^x[i + 1, j_x - 1])$  and
            $v' = (0, d_y, M_2^y[i + 1, i + 1])$ ;
47.     case 2:  $g_x = 0$ 
48.     Change the type of  $(u, v)$  into  $QQ$  in  $G_i$ ;
49.     The process of type  $QI$  is similar to that of  $IQ$ .
50.     for each edge  $(u, v)$  of type  $QQ$  in  $G_i$  do
51.       Let  $u = (0, d_x, M_2^x[i, i])$  and  $v = (0, d_y, M_2^y[i, i])$ ;
52.       EdgeCreation( $i, d_x, d_y, c$ );
53.   Return  $G_{i+1}$ ;

```

Theorem 1. *Algorithm 1 solves the two-string consensus problem under non-overlapping inversion and transposition distance in $O(n^5)$ time and $O(n^4)$ space.*

Proof. First, we prove the correctness of Algorithm 1 by showing that each matched pair of span length i is represented by some edge in G_i . We prove it below by induction on i .

Basis step: For $i = 0$, the matched pair of span length 0 is represented by an edge (u, v) in G_0 , where $d(u) = 0$ and $d(v) = 0$.

Induction step: For $i \geq 1$, we consider a matched pair Θ_x and Θ_y of span length i . Clearly, Θ_x and Θ_y are also a matched pair of span length $i - 1$ with $\ell(\mathcal{F}(\Theta_x, x, i)) = \ell(\mathcal{F}(\Theta_y, y, i))$. By induction hypothesis, a matched pair Θ_x and Θ_y corresponds to some edge (u, v) in G_{i-1} . At iteration $i - 1$, Algorithm 1 utilizes this edge to locate all candidate mutation fragments at column i on the mutation tables of x and y . By Observation 1, this implies that $\mathcal{F}(\Theta_x, x, i)$ and $\mathcal{F}(\Theta_y, y, i)$ are candidates identified by Algorithm 1. Since $\ell(\mathcal{F}(\Theta_x, x, i)) = \ell(\mathcal{F}(\Theta_y, y, i))$, there is some edge (u', v') in G_i to represent a matched pair Θ_x and Θ_y .

Next, we analyze the complexities of Algorithm 1. For convenience, we use $|D|$ to denote the number of edges

of type D in G_i . It can be verified that the Procedure 1 costs $O(n^2)$ time in the worst case. Procedure 2 is called at most $O(n)$ times and each time it can be done in $(|HH| + |HT| + |TH| + |HI| + |IH| + |TT| + |TI| + |IT| + |II|) * O(1) + (|QI| + |IQ| + |QH| + |HQ| + |QT| + |TQ|) * O(n) + |QQ| * O(n^2) = O(n^4) * O(1) + O(n^2) * O(n) + O(1) * O(n^2) = O(n^4)$. As a result, the total time complexity of Algorithm 1 is $O(n^5)$. In addition, the space complexity of Algorithm 1 is $O(|HH| + |HT| + |TH| + |HI| + |IH| + |TT| + |TI| + |IT| + |II|) = O(n^4)$. $\square$

4. Conclusions

In this work, we studied the two-string consensus problem under non-overlapping inversion and transposition distance, with applications especially in computational biology. In addition, we presented an $O(n^5)$ time and $O(n^4)$ space algorithm in the worst case to solve this problem. It is worth mentioning that the two-string consensus problem studied by Zohora and Rahman [9] considered only inversion and did not limit the number of inversions to be used. However, the two-string consensus problem we addressed in this work took account of both inversion and transposition, where the lengths of two substrings exchanged by a transposition are not necessarily identical, and the number of inversions and transpositions acting on each input string required not to exceed a given threshold. It will be interesting in the future to design more efficient algorithms to solve the two-string consensus problem under non-overlapping inversion and transposition distance.

References

- [1] A. Amir, H. Paryenty, L. Roditty, On the hardness of the consensus string problem, *Inf. Process. Lett.* 113 (2013) 371–374.
- [2] D.-J. Cho, Y.-S. Han, H. Kim, Alignment with non-overlapping inversions on two strings, in: S.P. Pal, K. Sadakane (Eds.), *WALCOM 2014*, in: LNCS, vol. 8344, 2014, pp. 261–272.
- [3] D.-J. Cho, Y.-S. Han, H. Kim, Alignment with non-overlapping inversions and translocations on two strings, *Theor. Comput. Sci.* 575 (2015) 90–101.
- [4] M. Frances, A. Litman, On covering problems of codes, *Theory Comput. Syst.* 30 (1997) 113–119.
- [5] J. Gramm, F. Huffner, R. Niedermeier, Closest strings, primer design, and motif search, in: L. Florea, B. Walenz, S. Hannenhalli (Eds.), *Currents in Computational Molecular Biology*, 2002, pp. 74–75, Poster abstracts of RECOMB.
- [6] Y.-L. Huang, C.L. Lu, Sorting by reversals, generalized transpositions, and translocations using permutation groups, *J. Comput. Biol.* 17 (2010) 685–705.
- [7] T.T. Ta, C.Y. Lin, C.L. Lu, An efficient algorithm for computing non-overlapping inversion and transposition distance, *Inf. Process. Lett.* 116 (2016) 744–749.
- [8] E.A. Yildirim, Two algorithms for the minimum enclosing ball problem, *SIAM J. Optim.* 19 (2008) 1368–1391.
- [9] F.T. Zohora, M.S. Rahman, Application of consensus string matching in the diagnosis of allelic heterogeneity, in: M. Basu, Y. Pan, J. Wang (Eds.), *ISBRA 2014*, in: LNBI, vol. 8492, 2014, pp. 163–175.